

Stochastic Trace Estimation with Hutchinson and Hutch++: A Reproducible Computational Study

Junguang Jia

Abstract

This study implements and evaluates two established stochastic trace estimators, Hutchinson and Hutch++, using a common matrix–vector interface. The experiments compare their accuracy on one positive semidefinite matrix with an exponentially decaying spectrum and on one synthetic graph, where triangle counting reduces to estimating $\text{tr}(A^3)/6$. The release supplies reproducible configurations, trial-level results, correctness tests, and a report generated from those results. It separates operator-query cost from measured runtime and reports Monte Carlo uncertainty conditional on each fixed input. The contribution is a documented computational replication and implementation study; it does not introduce a new estimator or establish performance across an input population.

1 Introduction

For an implicitly represented square matrix $M \in \mathbb{R}^{n \times n}$, stochastic trace estimation approximates $\text{tr}(M)$ using products $x \mapsto Mx$. It is useful when these products are available but obtaining the full matrix or its diagonal is expensive. An explicitly stored dense matrix already permits an exact trace in $O(n)$ operations; the dense example below is therefore a controlled accuracy benchmark, not a case where stochastic estimation is needed to compute the trace.

Hutchinson averages random quadratic forms [1]. Hutch++ combines a randomized low-rank approximation with a stochastic residual correction [2]. This study asks how these two implementations behave under the same maximum operator-query budget on two specified examples. The exponential-spectrum experiment follows the construction in the XTrace paper [3]; triangle counting is also an established application of trace estimation [2]. XTrace and XNysTrace motivate further comparisons but are outside this release.

2 Methods

2.1 Hutchinson

Let $z_1, \dots, z_m$ have independent Rademacher entries, each taking values -1 and $+1$ with equal probability. The estimator is

$$\hat{\tau}_H = \frac{1}{m} \sum_{j=1}^m z_j^\top M z_j.$$

Since $\mathbb{E}[zz^\top] = I$, $\mathbb{E}[z^\top Mz] = \text{tr}(M)$ for every square M . For a real symmetric matrix,

$$\text{Var}(\hat{\tau}_H) = \frac{2}{m} \sum_{i \neq j} M_{ij}^2 \leq \frac{2}{m} \|M\|_F^2.$$

For a positive semidefinite (PSD) matrix, $\|M\|_F \leq \text{tr}(M)$, so relative root-mean-square error has an $O(m^{-1/2})$ upper bound. This is a statement about RMS error; the empirical primary metric below is mean absolute relative error. For a diagonal matrix, Rademacher Hutchinson is exact.

2.2 Hutch++ and numerical rank

For a maximum budget m divisible by three, let $k = m/3$. Draw independent Rademacher matrices $S, G \in \mathbb{R}^{n \times k}$, form $Y = MS$, and compute an orthonormal basis $Q \in \mathbb{R}^{n \times r}$ for its numerical range using a thin singular value decomposition. Singular values at or below $\max(n, k) \epsilon_{\text{mach}} \sigma_1(Y)$ are discarded, where ϵ_{mach} is Float64 machine precision. With $P = QQ^\top$,

$$\hat{\tau}_{H++} = \text{tr}(Q^\top M Q) + \frac{1}{k} \sum_{j=1}^k g_j^\top (I - P) M (I - P) g_j.$$

Conditioning on Q leaves the columns of G independent and isotropic. Hence

$$\mathbb{E}[\hat{\tau}_{H++} \mid Q] = \text{tr}(MP) + \text{tr}((I - P)M(I - P)) = \text{tr}(M),$$

using $(I - P)^2 = I - P$ and cyclic invariance of the trace. Any orthonormal Q independent of G preserves this identity; the quality of the sketch affects variance rather than unbiasedness. Floating-point computation introduces the usual numerical approximation.

The implementation uses k products for MS , r for MQ , and k for the residual: $2k + r \leq m$ queries. Thus m is a maximum budget, with actual counts recorded separately. If one operator product costs T_{mv} , Hutchinson requires $O(mT_{\text{mv}} + mn)$ work. Hutch++ additionally incurs basis and projection work, with a bound $O(mT_{\text{mv}} + nk^2 + k^3)$ when $k \leq n$.

The PSD relative-error theorem for Hutch++ gives, with failure probability at most δ , an ϵ -relative approximation using $O(\epsilon^{-1} \sqrt{\log(1/\delta)} + \log(1/\delta))$ queries [2]. This theorem does not imply that Hutch++ is faster at every fixed budget, and the two examples here do not empirically establish an asymptotic convergence rate.

2.3 Triangle counting

For the adjacency matrix A of a simple undirected graph, each triangle gives six closed walks of length three. Therefore

$$T = \frac{\text{tr}(A^3)}{6}, \quad Mx = A(A(Ax)).$$

One query to $M = A^3$ costs three sparse adjacency-matrix products. Both estimators are applied to M and divided by six; the stochastic path never forms A^3 . An exact count supplies the reference answer. Although A^3 is symmetric, it need not be PSD, so the PSD relative-error theorem is not an accuracy guarantee for this experiment. Negative stochastic estimates are retained rather than clipped, which would change the estimator and introduce bias.

3 Experimental design

The full configuration specifies two fixed inputs:

- **Exponential PSD matrix.** Dimension $n = 1000$ and $M = U \text{diag}(\lambda) U^\top$, where $\lambda_i = 0.7^{i-1}$ and U is constructed by QR factorization of a seeded Gaussian matrix. The construction seed is 123 and the reference trace is $\sum_i \lambda_i$.
- **Synthetic graph.** A two-block stochastic block model with 400 vertices split equally between the blocks, within-block edge probability 0.08, between-block probability 0.01, and construction seed 2026. The graph has no self-loops and uses undirected, unweighted edges.

For each input, method, and budget $m \in \{6, 12, \dots, 120\}$, the accuracy experiment uses $R = 1000$ probe replications. Explicit random generators and recorded seeds define the trials independently of execution scheduling.

For a nonzero reference value τ , each trial records $e_j = |\hat{\tau}_j - \tau|/|\tau|$. The summary gives

$$\bar{e} = \frac{1}{R} \sum_{j=1}^R e_j, \quad \text{MCSE}(\bar{e}) = \frac{s_e}{\sqrt{R}},$$

where s_e is the sample standard deviation of the trial errors. This MCSE quantifies numerical Monte Carlo uncertainty about the mean error on a *fixed* input. It does not measure uncertainty across independently generated matrices or graphs. If the reference trace or triangle count is zero, relative error is undefined and is recorded as missing; absolute error remains available. It is never silently substituted into a relative-error series.

Runtime is assessed in a separate serial pass with one Julia thread and one BLAS thread. Each method uses the same vector-only operator interface. After three warm-up calls per case, thirty standalone calls are measured, including random-vector generation and estimator work but excluding input construction, exact-reference computation, random-generator construction, explicit pre-call garbage collection, and result export. Reported times are the median and interquartile range (IQR), not parallel throughput or summed per-vector instrumentation. Timings describe this implementation and machine.

4 Results

Table 1 and Figures 1–3 are generated from the saved experiment summaries. Lower error is better; error bars show one conditional MCSE on either side of the mean. The elapsed-time plot displays median runtime with its IQR. A method’s query efficiency and its runtime at a given budget are distinct observations. No inference of universal superiority is made from these two inputs.

At the largest tested budget, 120, the PSD mean relative errors are 4.301×10^{-2} for Hutchinson and 9.445×10^{-8} for Hutch++. Their median standalone runtimes are 1.441×10^{-2} and 1.560×10^{-2} seconds, respectively. On the fixed SBM graph, the corresponding mean relative errors are 9.863×10^{-2} and 4.203×10^{-2} . These values describe the saved run, conditional on its two fixed inputs.

Table 1: Selected accuracy summaries. $\bar{e}$ is mean absolute relative error; SE denotes its conditional Monte Carlo standard error.

Input	m	$\bar{e}_H$	SE_H	$\bar{e}_{H++}$	SE_{H++}
PSD	6	0.19	0.00438	0.2	0.00516
PSD	120	0.043	0.00105	9.44e-08	2.41e-09
SBM	6	0.439	0.0108	0.426	0.0111
SBM	120	0.0986	0.00232	0.042	0.00101

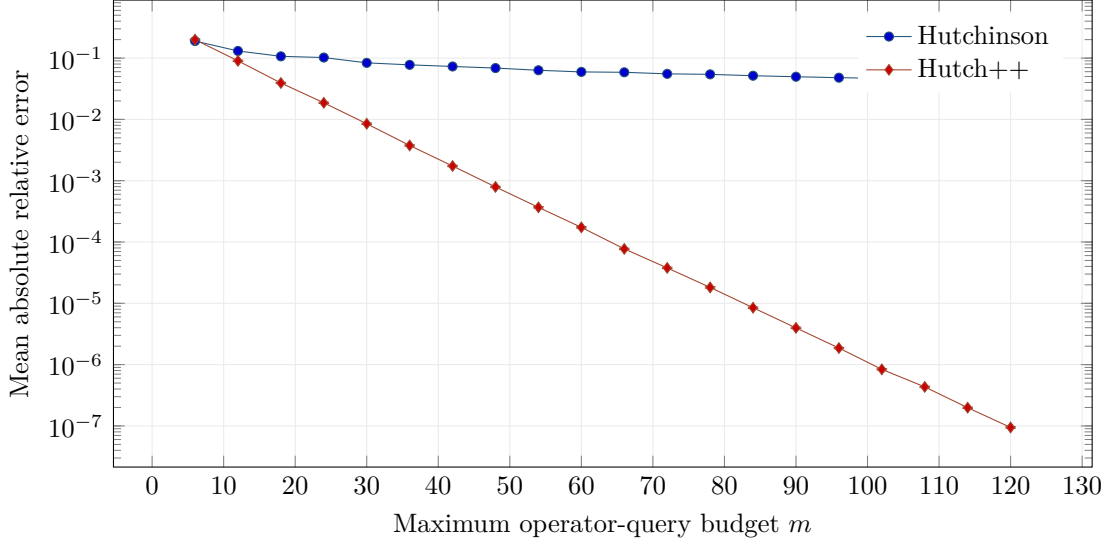


Figure 1: Accuracy on the fixed exponential-spectrum PSD matrix. Error bars show ± 1 MCSE, conditional on that matrix.

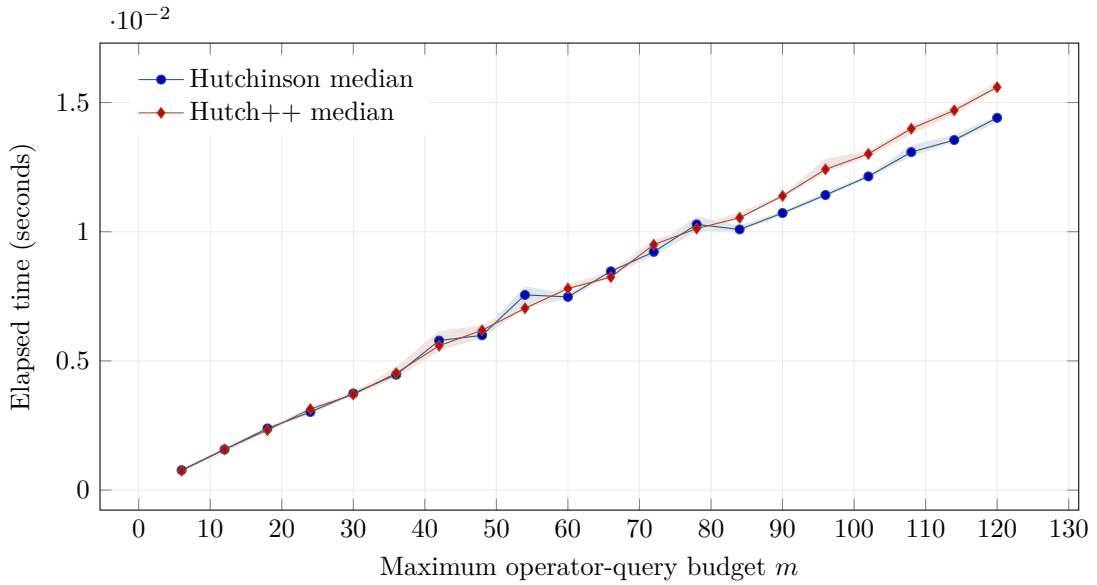


Figure 2: Standalone runtime on the PSD matrix. Bands show the 25th–75th percentiles across thirty timed calls after warm-up.

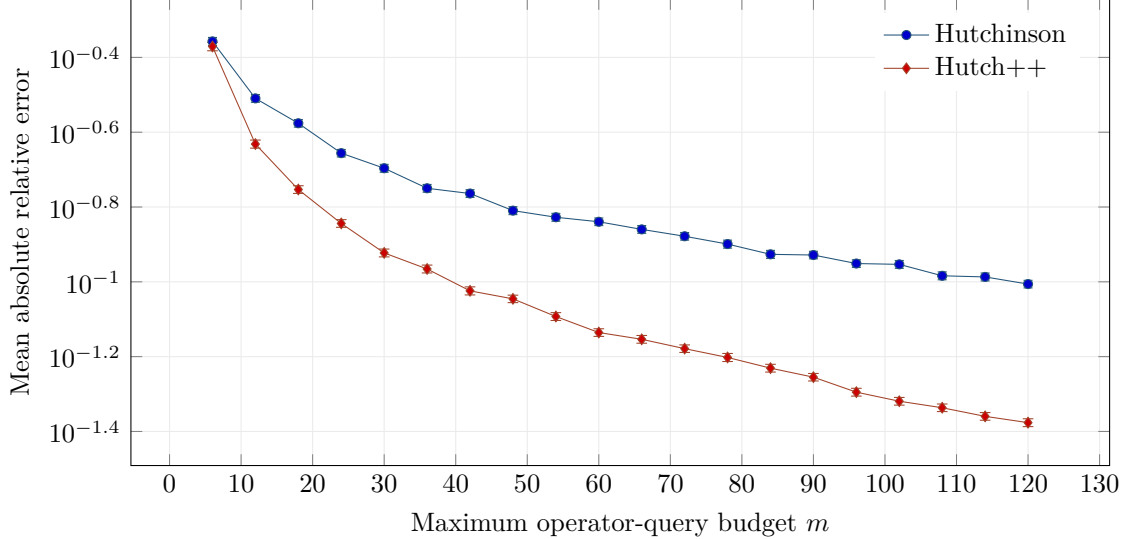


Figure 3: Triangle-count accuracy on the fixed SBM graph. Each operator query requires three sparse adjacency products; error bars show ± 1 conditional MCSE.

5 Limitations

There is one instance per example and no broad input-family or real-network benchmark. The exponentially decaying spectrum favors low-rank approximation; flatter spectra or different matrix orientations can change the comparison. A large number of probe replications does not remove this limitation. The graph is small enough to admit an exact reference calculation, and the experiment does not establish that stochastic counting outperforms graph-specific enumeration.

The report compares two established algorithms and does not assess the full range of modern trace estimators. Actual query counts can fall below the maximum budget after numerical-rank truncation. Floating-point arithmetic and the rank threshold affect very small residuals. Timing variability reflects local execution conditions; it is not a confidence interval for performance on other machines. Re-running the release may therefore change timings even when the accuracy trials are reproduced.

6 Reproducibility

The repository includes a Julia project, correctness tests, smoke and full experiment configurations, trial-level CSV outputs, summaries, and the script that embeds numeric results into this standalone $\LaTeX$ source. The README lists the exact commands. Smoke outputs validate the pipeline and must not be presented as the full study. Tests cover estimator identities, numerical-range handling, operator accounting, reproducibility, and graph reference checks; passing tests is distinct from empirical evidence of accuracy.

Run record. The saved run uses Julia 1.12.5, Darwin on aarch64, with 1 Julia thread and 1 BLAS thread. The configuration is `full`. The reference PSD trace is 3.3333333333; the generated graph has 1290 triangles. The run completed at 2026-09-27T04:09:49.839 UTC. Exact package versions and the timing protocol are recorded in `metadata.toml`.

Source fingerprint (SHA-256). The build records the fingerprint of its declared source inputs, allowing results to be matched to the generating implementation. This fingerprint does not assert bitwise reproducibility of elapsed times.

f5dad64af5fca6453ab6aa0f84f2393c84f29b24820f5968678cf4ee84e79500

References

- [1] M. F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics—Simulation and Computation*, 18(3):1059–1076, 1989. doi:10.1080/03610918908812806.
- [2] R. A. Meyer, C. Musco, C. Musco, and D. P. Woodruff. Hutch++: Optimal stochastic trace estimation. *Symposium on Simplicity in Algorithms (SOSA)*, pp. 142–155, 2021. arXiv:2010.09649.
- [3] E. N. Epperly, J. A. Tropp, and R. J. Webber. XTrace: Making the most of every sample in stochastic trace estimation. *SIAM Journal on Matrix Analysis and Applications*, 45(1):1–23, 2024. doi:10.1137/23M1548323.